

## ЛЕКЦИЯ 4

# Структурный анализ сетей Петри и беступиковое распределение ресурсов

*Почему ячейка встала — и как доказать структурно, что она не встанет*

две части по 45

ение робототехническими комплексами»

# План занятия

- 1 Что даёт сеть Петри сверх автомата
- 2 Уравнение состояния и ложные решения
- 3 Р-инварианты: доказательство свойств без перебора
- 4 Взаимное исключение и ёмкость буфера как теоремы
- 5 Т-инварианты и производственные циклы
- 6 Живость, ограниченность, обратимость
- 7 Сифоны и ловушки: структурная причина тупика
- 8 Системы распределения ресурсов  $S^3PR$
- 9 Три способа синтеза беступиковой политики
- 10 Методика анализа ячейки

*Синие — первая часть · янтарные — вторая часть*

ЧАСТЬ 1

# Алгебра вместо перебора

*Как доказывать свойства системы, не строя множества её состояний.*

### Автомат

- Состояние — элемент множества  $X$
- 5 паллет в 4 местах:  $4^5 = 1024$  состояния
- Структура зависит от числа объектов
- Анализ — обход графа

### Сеть Петри

- Состояние — вектор маркировки  $M \in \mathbb{N}^{|P|}$
- 5 паллет в 4 местах: 4 позиции, число фишек — параметр
- Структура не зависит от числа объектов
- Анализ — линейная алгебра над матрицей инцидентности

“

Сеть Петри позволяет доказать свойство сразу для всех достижимых состояний, решив систему линейных уравнений.

*Полиномиальное время вместо экспоненциального перебора*

$$M = M_0 + C \cdot \sigma^{\rightarrow}$$

- $C = C^+ - C^-$  — матрица инцидентности
- $\sigma^{\rightarrow}$  — вектор Париха: сколько раз сработал каждый переход
- Доказывается индукцией по длине последовательности
- Условие достижимости **необходимое**, но не достаточное

Ложные решения: уравнение выполнено, а последовательность нереализуема

$$y^T C = 0 \Rightarrow y^T M = y^T M_0 \text{ для всех } M \in R(N, M_0)$$

- Доказательство: умножаем уравнение состояния слева на  $y^T$
- Взвешенная сумма фишек постоянна на всех достижимых маркировках
- Инварианты — ядро системы  $C^T y = 0$ , вычисляются за полиномиальное время
- Алгоритм Фаркаша даёт минимальные неотрицательные инварианты

## 4.4

## Два следствия, которые нужны каждый день

- **Ограниченность:** если  $y \geq 0$  и  $y(p) > 0$ , то  $M(p) \leq \lfloor y^T M_0 / y(p) \rfloor$
- Буфер не переполнится — доказано, а не протестировано
- **Взаимное исключение:**  $M(p_R) + \sum M(p_{use}, i) = 1$
- Сумма неотрицательных целых равна 1  $\Rightarrow$  не более одного слагаемого равно 1
- Ресурс используется не более чем одним потребителем — теорема

### Ключевая идея

Это не рассуждение «мы поставили семафор», а математическое утверждение обо всех достижимых состояниях. Если такого инварианта нет, взаимное исключение не гарантировано — и это обнаруживается решением линейной системы.

## ? Проверим

В сети найден Р-инвариант  $M(\text{free}) + M(\text{use}_1) + M(\text{use}_2) + M(\text{use}_3) = 2$ . Что это означает физически?

**A** Ресурс используется не более чем двумя потребителями одновременно

**B** В системе ровно два потребителя

**C** Буфер вмещает две детали

**D** Ничего определённого

## ? Проверим

В сети найден Р-инвариант  $M(\text{free}) + M(\text{use}_1) + M(\text{use}_2) + M(\text{use}_3) = 2$ . Что это означает физически?

**A** Ресурс используется не более чем двумя потребителями одновременно

**B** В системе ровно два потребителя

**C** Буфер вмещает две детали

**D** Ничего определённого

### ОТВЕТ

Первое: ресурс имеет ёмкость 2, и в любой достижимой маркировке занято не более двух экземпляров. Число потребителей при этом может быть любым.

- $C \cdot x = 0$ : последовательность, возвращающая систему в исходную маркировку
- Каждый минимальный Т-инвариант — один законченный технологический маршрут
- Компоненты вектора — число срабатываний операций за цикл
- Переход вне носителя любого Т-инварианта не участвует в цикле
- Это структурный признак ошибки моделирования

| Свойство           | Определение                                | Инженерный смысл                      |
|--------------------|--------------------------------------------|---------------------------------------|
| Ограниченность     | $M(p) \leq k$ для всех достижимых $M$      | буфер не переполняется                |
| Живость            | каждый переход когда-нибудь снова возможен | ни одна операция не выпадает навсегда |
| Отсутствие тупиков | нет маркировки без разрешённых переходов   | система не встаёт                     |
| Обратимость        | $M_0$ достижима из любой $M$               | возврат в исходное состояние возможен |

*Живость сильнее отсутствия тупиков: система может работать вечно, но некоторая операция никогда не выполняться*

- **Сифон**  $S$ :  $\bullet S \subseteq S \bullet$  — всякий переход, кладущий в  $S$ , берёт из  $S$
- **Ловушка**  $Q$ :  $Q \bullet \subseteq \bullet Q$  — всякий переход, берущий из  $Q$ , кладёт в  $Q$
- Понятия двойственны: сифон в  $N$  есть ловушка в сети с обращёнными дугами
- Опустевший сифон пуст навсегда
- Помеченная ловушка помечена навсегда

### Ключевая идея

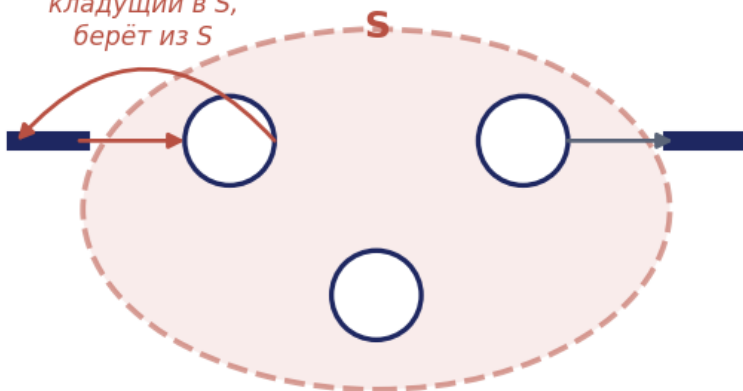
Обе теоремы доказываются в три строки разбором двух случаев на один шаг срабатывания. Это тот случай, когда простое рассуждение даёт мощный инструмент диагностики.

### Сифон и ловушка: двойственные структурные понятия

**Сифон S:** вход(S) содержится в выход(S)

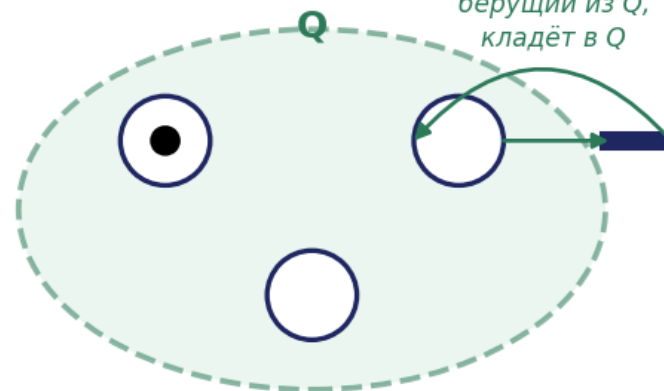
**Ловушка Q:** выход(Q) содержится в вход(Q)

всякий переход,  
кладеющий в S,  
берёт из S



**Опустевший сифон  
пуст навсегда**

всякий переход,  
берущий из Q,  
кладёт в Q



**Помеченная ловушка  
помечена навсегда**

Всякая тупиковая маркировка содержит пустой сифон - структурная причина, а не только факт блокировки

Опустевший сифон пуст навсегда; помеченная ловушка помечена навсегда. Доказательства - в три строки.

**М — тупик  $\Rightarrow$  множество пустых позиций содержит пустой сифон**

- Всякий переход имеет входную позицию среди пустых (иначе он был бы разрешён)
- Итеративное сужение множества пустых позиций даёт сифон
- Непустота доказывается от противного по порядку удаления
- Сифон **называет причину**: какие ресурсы исчерпались совместно

Диагностика тупика превращается из гадания в чтение результата анализа

- Если каждый сифон содержит помеченную ловушку — тупиков нет
- Доказательство: сифон пуст  $\Rightarrow$  ловушка внутри него пуста — противоречие
- Для сетей свободного выбора условие ещё и **необходимо** для живости
- Полный структурный критерий без построения графа достижимости
- Но: сети РТК с разделяемыми ресурсами свободным выбором не обладают

### Факт

Именно смешение конкуренции за разные ресурсы разрушает свойство свободного выбора — и оно же порождает тупики. Для таких сетей развита отдельная теория.

## ? Найдём сифон

Процесс 1 берёт робота, потом зону. Процесс 2 берёт зону, потом робота. Какое множество образует опасный сифон?

A

Только ресурсные позиции  $\{r_M, r_Z\}$

B

$\{r_M, r_Z\}$  плюс операционные позиции, где удерживаются оба

C

Все позиции сети

D

Позиции простоя обоих процессов



## Найдём сифон

Процесс 1 берёт робота, потом зону. Процесс 2 берёт зону, потом робота. Какое множество образует опасный сифон?

A

Только ресурсные позиции  $\{r_M, r_Z\}$

B

$\{r_M, r_Z\}$  плюс операционные позиции, где удерживаются оба

C

Все позиции сети

D

Позиции простоя обоих процессов

### ОТВЕТ

Второе:  $S = \{r_M, r_Z, a_2, b_2\}$ . Все переходы, кладущие фишку в эти позиции, берут фишку из них же — условие сифона выполнено, и при взаимной блокировке  $S$  пуст.

П Е Р Е Р Ы В

## Конец первой части

*Мы научились находить структурную причину тупика. После перерыва — как построить политику, гарантирующую его отсутствие.*

## ЧАСТЬ 2

# Как гарантировать отсутствие тупиков

*Три метода, их цена и критерии выбора для конкретной ячейки.*

## Класс $S^3PR$ : модель производственной ячейки

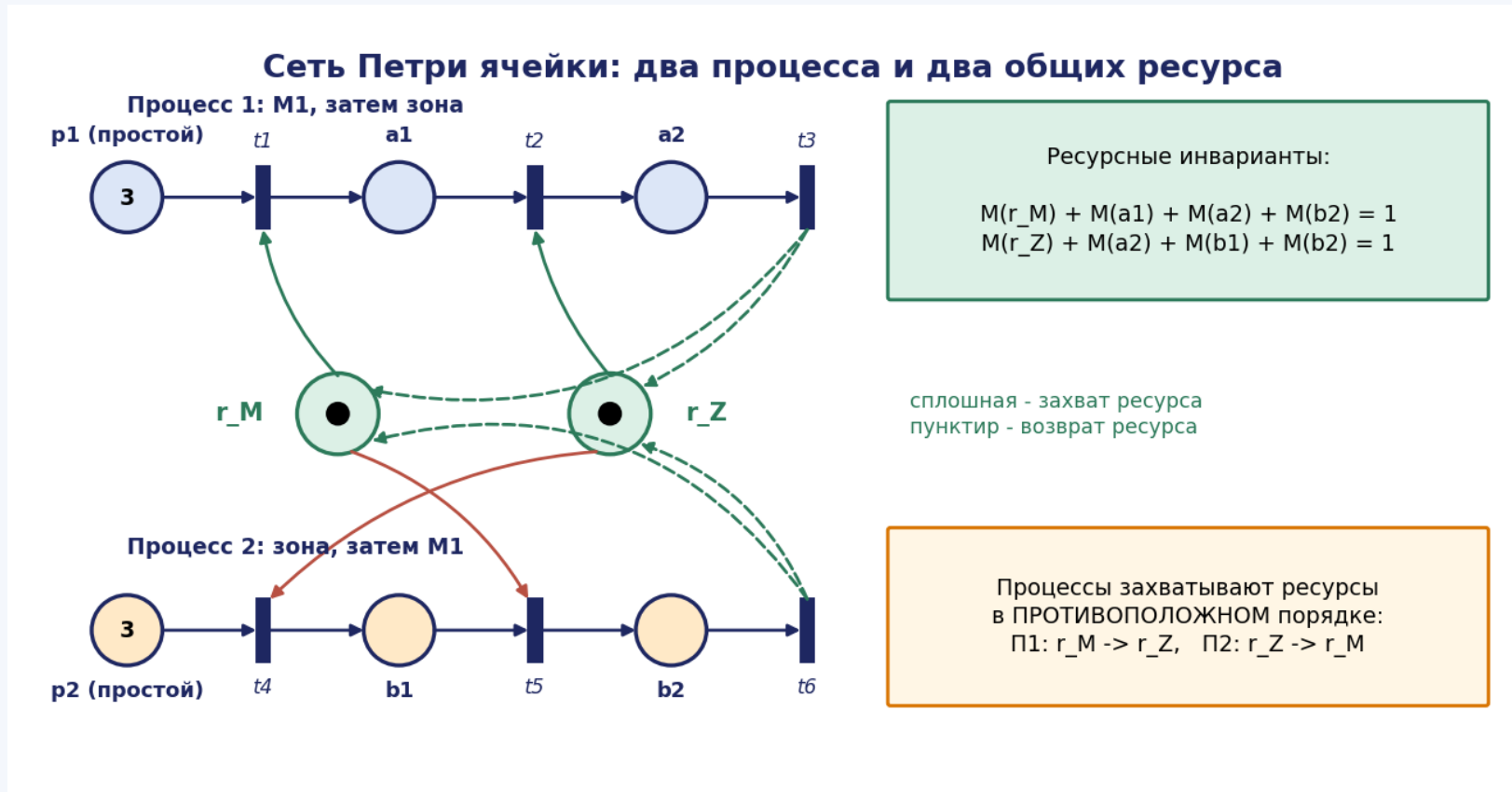
- Каждый процесс — простой последовательный маршрут через позицию простоя
- Каждая операция требует ровно один ресурс, удерживаемый на время операции
- Ресурсы общие для разных процессов
- Схема «удержание и ожидание» из главы 1 возникает по построению
- Ресурсный инвариант:  $M(r) + \sum_{p \in H(r)} M(p) = C(r)$

### Ключевая идея

Живость  $S^3PR$  эквивалентна невозможности опустошения строгих минимальных сифонов. Проверка сводится к анализу конечного множества структурных объектов.

## 4.10

## Сеть Петри ячейки класса S3PR



Процессы захватывают ресурсы в противоположном порядке - структурная причина будущего тупика.

## 4.11 Разбор тупика в ячейке



Причина названа: противоположный порядок захвата двух ресурсов

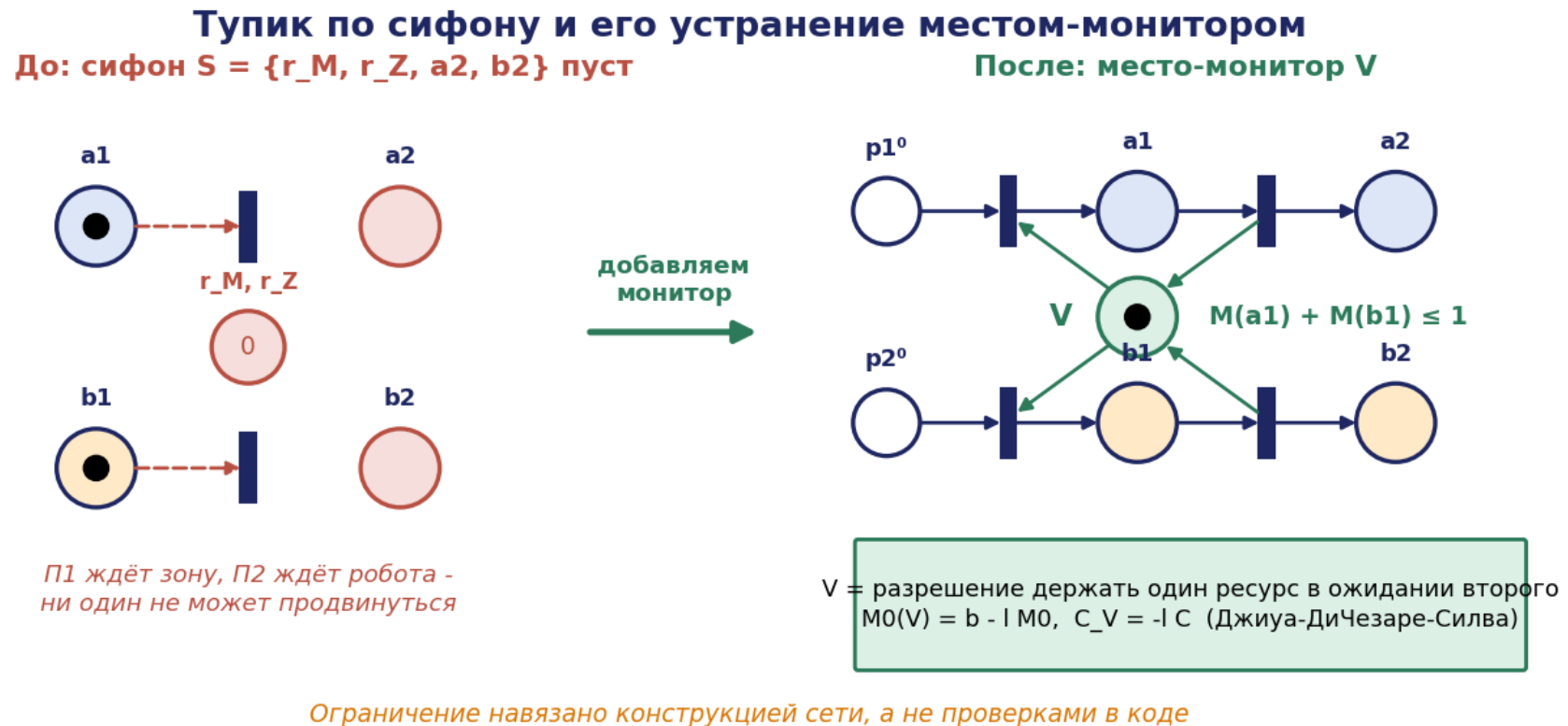
## Метод 1: место-монитор

$$M_0(V) = \beta - I^T M_0, \quad C_V = -I^T C$$

- Реализует ограничение  $I^T M \leq \beta$  добавлением одной позиции
- Доказательство: вектор  $(I, 1)$  — Р-инвариант расширенной сети
- Из  $M(V) \geq 0$  следует  $I^T M \leq \beta$  на всех достижимых маркировках
- Ограничение навязано конструкцией сети, а не проверками в коде

Монитор физически соответствует счётчику разрешений с вычисляемой начальной маркировкой

## 4.12 Тупик и его устранение монитором



Ограничение навязано конструкцией сети, а не проверками в коде.

## 4.13 Метод 2: банкирский алгоритм

- Состояние безопасно, если существует порядок завершения всех процессов
- Проверка: искать процесс, чьи потребности покрываются свободными ресурсами
- Завершив его, вернуть ресурсы и повторить
- Запрос удовлетворяется, только если результат безопасен
- Сложность  $O(k^2 \cdot m)$  на каждый запрос

### Ключевая идея

Метод максимально разрешителен: он не запрещает состояния структурно, а проверяет каждый запрос. Цена — необходимость знать оставшиеся потребности процессов заранее.

## 4.14 Метод 3: упорядочивание ресурсов

- Ввести полный порядок на ресурсах
- Требовать захвата строго в возрастающем порядке
- Круговое ожидание становится невозможным
- Доказательство: цикл ожидания даёт  $r_1 < r_2 < \dots < r_1$  — противоречие
- Устраняет четвёртое условие Коффмана структурно

### Пример

Для разобранного тупика достаточно потребовать порядок  $(r_M, r_Z)$  для обоих процессов. Но это изменение технологического маршрута, и оно должно быть согласовано с технологом, а не введено программистом молча.

## 4.15 Сравнение трёх методов

| Критерий             | Мониторы            | Банкир                 | Порядок              |
|----------------------|---------------------|------------------------|----------------------|
| Момент решения       | проектирование      | исполнение             | проектирование       |
| Стоимость исполнения | нулевая             | $O(k^2m)$ на запрос    | нулевая              |
| Разрешительность     | консервативнее      | максимальная           | зависит от маршрутов |
| Верифицируемость     | полная, структурная | требует доверия к коду | полная               |
| Гибкость маршрутов   | требует пересинтеза | поддерживается         | ограничивает         |

*Жёсткая ячейка — мониторы; гибкая система с изменяющимися маршрутами — банкир*



## Выбираем метод

Ячейка выпускает одну номенклатуру по фиксированному маршруту, вычислитель загружен на 80 %. Какой метод предпочесть?

А

Банкирский алгоритм — он оптимален

В

Места-мониторы — нулевая стоимость исполнения

С

Упорядочивание ресурсов — проще всего

Д

Никакой — при одной номенклатуре тупиков не бывает



## Выбираем метод

Ячейка выпускает одну номенклатуру по фиксированному маршруту, вычислитель загружен на 80 %. Какой метод предпочесть?

A

Банкирский алгоритм — он оптимален

B

Места-мониторы — нулевая стоимость исполнения

C

Упорядочивание ресурсов — проще всего

D

Никакой — при одной номенклатуре тупиков не бывает

### ОТВЕТ

Мониторы: маршрут фиксирован, значит консервативность известна и приемлема, а исполнение бесплатно при загруженном вычислителе. Четвёртый вариант неверен — тупики возникают из параллелизма, а не из номенклатуры.

## 4.16 Вычислительная сложность задач

| Задача                       | Сложность                                                      |
|------------------------------|----------------------------------------------------------------|
| Вычисление Р/Т-инвариантов   | полиномиальная (число минимальных может расти экспоненциально) |
| Проверка ограниченности      | EXPSPACE                                                       |
| Достижимость маркировки      | разрешима, Ackermann-полна                                     |
| Живость                      | EXPSPACE-трудна                                                |
| Существование пустого сифона | NP-полна                                                       |

*Отсюда порядок работы: сначала дешёвая линейная алгебра, потом структура, перебор — в крайнем случае*

## 4.17 Методика анализа ячейки



Шаг 5 обязателен: добавление мониторов меняет структуру и может породить новые сифоны

## 4.18 Что даёт структурный анализ: цифры

**ПОЛИНОМ**

время вычисления инвариантов

**все**

достижимые маркировки  
покрыты одним  
доказательством

**1**

формула для начальной  
маркировки монитора

**4**

условия Коффмана, из них  
третье и четвёртое устранимы

- Модель параметрична: число паллет — параметр, а не структура
- Сифон называет причину, а не только факт блокировки

## 4.19 Инструменты

| Инструмент | Возможности                                               |
|------------|-----------------------------------------------------------|
| TINA       | инварианты, граф достижимости, временные сети, LTL        |
| PIPE       | редактор, инварианты, GSPN, удобен для обучения           |
| CPN Tools  | цветные и временные сети, анализ пространства состояний   |
| LoLA       | высокопроизводительная верификация больших сетей          |
| INA        | классический анализ структурных свойств, сифоны и ловушки |

На семинаре 3 — PIPE плюс собственная реализация поиска сифонов

### Супервизор (гл. 3)

- Явное состояние
- Максимальная разрешительность доказана
- Учёт неуправляемости встроен
- Контрпример — трасса
- Ограничен размером  $|X|$

### Структурный анализ (гл. 4)

- Структурное состояние — вектор
- Обычно консервативен
- Неуправляемость требует расширения
- Контрпример — сифон с именами ресурсов
- Параметричен по числу фишек

## 4.21 Типовые ошибки при построении сети

- Ресурс не выделен в отдельную позицию — инвариант не находится
- Операция моделируется одним переходом вместо позиции с двумя переходами
- Ёмкость буфера задана комментарием, а не структурой
- Отказы не включены — анализируется номинальный режим
- Начальная маркировка выбрана «для примера» и не соответствует пуску

### Факт

Первые две ошибки обнаруживаются автоматически: ожидаемый Р-инвариант не находится, а Т-инварианты не покрывают все переходы. Это самая дешёвая проверка модели.

“

Тупик — не случайность и не «сбой». Это структурное свойство, у которого есть имя, причина и вычислимое условие отсутствия.

*Именно поэтому с ним можно работать инженерно*

# Выводы

- 1 Сеть Петри представляет состояние структурно — модель параметрична по числу однотипных объектов
- 2 Уравнение состояния даёт необходимое условие: доказывает невозможность, но не возможность
- 3 Р-инварианты доказывают ограниченность и взаимное исключение за полиномиальное время
- 4 Опустевший сифон пуст навсегда; всякий тупик содержит пустой сифон
- 5 Для  $S^3PR$  живость эквивалентна невозможности опустошения строгих минимальных сифонов
- 6 Три метода: мониторы (дёшево, консервативно), банкир (оптимально, дорого), порядок (просто, ограничивает)
- 7 Отсутствие ожидаемого инварианта — признак ошибки моделирования, а не системы

# Вопросы для самопроверки

1. Почему уравнение состояния даёт лишь необходимое условие достижимости?
2. Докажите постоянство взвешенной суммы по Р-инварианту
3. Как из Р-инварианта получить границу ёмкости буфера?
4. Дайте определения сифона и ловушки; в чём их двойственность?
5. Докажите, что опустевший сифон остаётся пустым
6. Почему теорема о пустом сифоне в тупике ценна для диагностики?
7. Почему условие Коммонера достаточно, но в общем случае не необходимо?
8. Что такое строгий минимальный сифон и почему он централен в теории  $S^3PR$ ?
9. Сравните три метода по разрешительности и стоимости исполнения
10. Почему добавление места-монитора может породить новый сифон?